FULL STACK DEVELOPER ROADMAP

Beginner to Industry-Oriented MERN Stack Learning Plan

FRONTEND	BACKEND	DATABASE
HTML • CSS • JavaScript React • TypeScript	Node.js • Express.js REST APIs • Authentication	MongoDB • Mongoose SQL fundamentals

Your end goal: Learn to plan, build, test and deploy complete web applications—from the user interface to server logic and database.

Suggested duration: Approximately 8 months at 2–3 focused hours per day. Adjust the pace to your college workload and actual progress.

HOW TO USE THIS ROADMAP

1. Follow the phases in order. 2. Write code yourself instead of only watching tutorials. 3. Build each project without copying a complete solution. 4. Push working versions to GitHub. 5. Review and improve your code.

MERN = MongoDB + Express.js + React.js + Node.js. JavaScript is used across the frontend and backend in this stack.

1. Technology Map

AREA	CORE TECHNOLOGIES	WHAT YOU BUILD
Frontend	HTML5, CSS3, JavaScript	Pages, layouts, forms, responsive interfaces
Frontend advanced	React, TypeScript, Tailwind CSS	Reusable components and interactive applications
Backend	Node.js, Express.js	Servers, routes, middleware and business logic
Database	MongoDB, Mongoose, SQL basics	Store, query and manage application data
API & security	REST, JSON, validation, JWT, bcrypt	Connect clients securely to server functionality
Engineering	Git, GitHub, testing, Docker basics, CI/CD	Collaborate, verify, package and ship software
Deployment	Vercel, Render and cloud fundamentals	Publish and monitor live applications

2. Eight-Month Learning Plan

PHASE 1 — WEB FOUNDATIONS | Weeks 1–4

• HTML: document structure, semantic elements, links, images, tables and forms.
• CSS: selectors, box model, typography, Flexbox, Grid, responsive design and basic animation.
• JavaScript: variables, data types, operators, conditions, loops, functions and arrays.

Build: Portfolio website, responsive landing page and calculator.

PHASE 2 — JAVASCRIPT PRACTICE | Weeks 5–8

• ES6+: let/const, arrow functions, destructuring, spread/rest, modules and array methods.
• DOM manipulation, events, form validation and browser storage.
• Promises, async/await, Fetch API, JSON and error handling.

Build: Quiz app, weather app using an API and expense tracker.

PHASE 3 — REACT + TYPESCRIPT | Weeks 9–14

• React components, props, state, events, hooks and forms.
• React Router, Context API, reusable components and API integration.
• TypeScript basics: types, interfaces, unions, generics and typing React components.
• Tailwind CSS for consistent, responsive UI styling.

Build: Task dashboard and e-commerce frontend.

PHASE 4 — BACKEND DEVELOPMENT | Weeks 15–20

• Node.js runtime, modules, npm, environment variables and asynchronous I/O.
• Express.js routes, controllers, middleware and request/response lifecycle.
• REST API design, HTTP methods, status codes, validation and centralized error handling.
• Authentication and authorization concepts; secure password hashing and session/token handling.

Build: Authentication API and blog API.

PHASE 5 — DATABASES | Weeks 21–24

• MongoDB documents, collections, CRUD operations and query filters.
• Mongoose schemas, models, validation, references and population.
• Indexes, aggregation basics, backups and database access control.
• SQL fundamentals: tables, keys, joins and basic PostgreSQL queries.

Build: Student management system and inventory management system.

PHASE 6 — SOFTWARE ENGINEERING & SHIPPING | Weeks 25–32

• Testing: unit, integration and API tests; learn Jest/Vitest and Supertest.
• Security: input validation, access control, safe configuration and common web risks.
• Git branches, pull requests, code reviews and useful README documentation.
• Docker fundamentals, CI/CD concepts, deployment, logs and monitoring basics.

Build: Deploy a complete full-stack application with documented setup and tests.

3. Project Ladder

Build projects in increasing complexity. Each project should have a clear README, screenshots, meaningful Git commits and a deployed version when practical.

LEVEL	PROJECT	FEATURES TO PRACTICE
Beginner	Personal portfolio	Semantic HTML, responsive CSS, navigation and contact form UI
Beginner	Calculator / To-do app	JavaScript functions, events, DOM and local storage
Intermediate	Weather app	Fetch API, async/await, loading and error states
Intermediate	Expense tracker	Forms, validation, state and data persistence
Intermediate	Blog application	React UI, REST API, CRUD and database models
Advanced	E-commerce platform	Products, cart, user accounts, orders and admin roles
Advanced	Real-time chat app	WebSockets, authentication and message persistence
Advanced	Learning management system	Roles, courses, lessons, progress and file handling
Industry-style	Multi-user SaaS app	Tenant-aware data, permissions, billing concepts, tests and monitoring

4. Weekly Practical Routine

DAY	FOCUS	OUTPUT
Monday	Learn a new concept	Short notes + small examples
Tuesday	Coding exercises	Solve problems without copying
Wednesday	Build a feature	One working component or API route
Thursday	API / database practice	Connect data and test responses
Friday	Debugging and review	Fix bugs and refactor code
Saturday	Mini-project day	Finish and document a feature
Sunday	Revision + GitHub	Review, commit and plan next week

Suggested commitment: 2–3 focused hours per day. On busy college days, do a shorter coding session rather than abandoning the routine.

5. Industry Readiness Checklist

Use this checklist to judge your actual skills—not just whether you have watched a course.

- ■ I can build a responsive frontend from a design or written requirements.
- ■ I understand JavaScript asynchronous code and can debug it.
- ■ I can build reusable React components and type them with TypeScript.
- ■ I can design REST APIs with validation and consistent error responses.
- ■ I can model application data and write useful MongoDB queries.
- ■ I understand authentication, authorization and basic web security.
- ■ I can write and run unit and API integration tests.
- ■ I can use Git branches, commits and pull requests.
- ■ I can deploy a frontend and backend and configure environment variables safely.
- ■ I can explain the architecture and trade-offs in my own projects.

6. Core CS Skills for Software Engineering Interviews

Data Structures & Algorithms: arrays, strings, hashing, linked lists, stacks, queues, trees, graphs, heaps, sorting, searching, recursion, backtracking, greedy algorithms and dynamic programming.

Computer Science Fundamentals: OOP, DBMS, SQL, operating systems, computer networks, HTTP, browser basics and time/space complexity.

Engineering habits: readable code, debugging, documentation, testing, code review and explaining technical decisions.

7. Final Milestone

YOUR PORTFOLIO SHOULD SHOW

- Strong frontend fundamentals and at least one polished responsive UI
- Backend APIs connected to a real database
- Authentication and role-based access where relevant
- Automated tests for important flows
- Several well-documented projects on GitHub
- At least one complete, deployed full-stack application

Remember: the eight-month schedule is a suggested path, not a guarantee of employment or internship placement. Your real progress is measured by what you can build, explain, test and improve independently.

BUILD. TEST. DEPLOY. IMPROVE.

Become a developer who can take an idea from an empty folder to a working product.